CURSO

Introduccion a python

Público: Tutores y formadores independientes

Duración: 12-16 horas

Introduccion a python

Módulo 1: Python Básico

Breve introducción al módulo: Este módulo establece los cimientos esenciales de Python, enfocados en la práctica inmediata para que tutores y formadores independientes puedan comenzar a escribir scripts útiles desde la primera sesión.

1.1 Sintaxis introductoria

Objetivos de aprendizaje: Comprender la estructura básica de un script en Python, utilizar comentarios para documentar código y emplear la función `print` para mostrar información en consola.
Explicación desarrollada: Python es un lenguaje de interpretación donde la indentación define los bloques de código. Los scripts comienzan con una línea de shebang opcional y se ejecutan en orden secuencial. Los comentarios, iniciados con el símbolo `#`, no afectan la ejecución pero mejoran la legibilidad. La función `print()` muestra texto o valores en la salida estándar.
Puntos clave: Uso de `#` para comentarios; llamada a `print()` con cadenas de texto; estructura mínima de un script ejecutable.
Ejemplo práctico: Un tutor crea un script que imprime un mensaje de bienvenida a sus alumnos al iniciar una sesión virtual:

```
# Script de bienvenida
print("Bienvenidos a la sesión de Python para tutores")
```

Duración sugerida: 2-2 horas

1.2 Variables y tipos de datos

Objetivos de aprendizaje: Declarar y asignar variables, identificar los tipos de datos básicos (cadena, número entero, número decimal, booleano) y realizar conversiones de tipo simples.
Explicación desarrollada: En Python, las variables se crean mediante asignación (`=`) y no requieren declaración de tipo explícita; el tipo se determina por el valor asignado. Los tipos más comunes son `str` (cadena de texto), `int` (entero), `float` (decimal) y `bool` (booleano). Se pueden usar funciones como `str()`, `int()` y `float()` para convertir entre tipos.
Puntos clave: Asignación simple; tipos `str`, `int`, `float`, `bool`; conversión de tipo con funciones integradas.
Ejemplo práctico: Un tutor guarda la duración estimada de una actividad en una variable entera y el nombre del curso en una cadena para personalizar un mensaje automático:

```
duracion_minutos = 45
nombre_curso = "Introducción a Python"
mensaje = f"El curso {nombre_curso} tiene una duración estimada de {duracion_minutos}
minutos"
print(mensaje)
```

Duración sugerida: 2-2 horas

Taller de práctica Módulo 1:
1. Escribe un script que imprima tu nombre completo usando `print`.
2. Asigna tu nombre a una variable llamada `nombre` e imprime un saludo personalizado usando `f-string`.
3. Añade un comentario en la segunda línea explicando qué hace la variable `nombre`.

Respuestas orientativas:
1. `print("Tu Nombre")`
2. `nombre = "Tu Nombre"; print(f"Hola, {nombre}!")`
3. Comentario: `# Aquí almaceno el nombre del tutor para usarlo en saludos`

Cierre del Módulo 1: En este módulo cubrimos la estructura básica de scripts, el uso de comentarios y la declaración de variables con sus tipos fundamentales. Estos conceptos son la base para cualquier programa en Python y permitirán automatizar tareas rutinarias en el contexto docente. En el próximo módulo exploraremos cómo tomar decisiones y repetir acciones con estructuras de control.

Módulo 2: Estructuras de Control

Breve introducción al módulo: Aprenderás a controlar el flujo de tus programas mediante condiciones y bucles, esenciales para procesar datos variables y crear flujos de trabajo adaptables en tareas pedagógicas.

2.1 Condicionales y bucles

Objetivos de aprendizaje: Utilizar estructuras `if`, `elif` y `else` para toma de decisiones, y emplear bucles `for` y `while` para repetir acciones.
Explicación desarrollada: Las condicionales permiten ejecutar bloques de código diferentes según si una expresión booleana es verdadera o falsa. En Python, `if`, `elif` y `else` deben ir seguidos de un dos puntos y la indentación es obligatoria. Los bucles `for` iteran sobre secuencias (como listas o rangos), mientras que `while` se ejecuta mientras una condición permanezca verdadera. Se pueden usar `break` y `continue` para controlar la iteración.
Puntos clave: Sintaxis de `if/elif/else` con indentación; bucles `for` sobre `range()` y secuencias; `while` con condición de parada; `break` y `continue`.
Ejemplo práctico: Un tutor escribe un script que verifica si un alumno ha completado los requisitos previos y muestra un mensaje u otro según el caso:

```
tiene_prerrequisitos = True

if tiene_prerrequisitos:
    print("Acceso concedido a la actividad avanzada.")
else:
    print("Por favor, completa los requisitos previos primero.")
```

Duración sugerida: 2-2 horas

2.2 Listas y diccionarios

Objetivos de aprendizaje: Crear y modificar listas, acceder a elementos por índice o clave, y utilizar diccionarios para asociar valores a identificadores únicos.
Explicación desarrollada: Las listas son colecciones ordenadas y mutables de elementos, accesibles por índice numérico y admiten operaciones de slicing. Los diccionarios almacenan pares clave-valor, donde cada clave debe ser única y suele ser de tipo `str`; permiten búsquedas rápidas y estructuración de datos relacionados. Ambos tipos admiten métodos comunes como `.append()`, `.pop()`, `.get()` y `.keys()`.
Puntos clave: Definición de listas `[]` y diccionarios `{};` indexación y slicing en listas; acceso a valores por clave en diccionarios; métodos `.append()`, `.get()`.
Ejemplo práctico: Un tutor usa una lista para almacenar los nombres de sus grupos de trabajo y un diccionario para mapear cada grupo a su horario de sesiones:

```
grupos = ["Grupo A", "Grupo B", "Grupo C"]
horarios = {"Grupo A": "Lunes 10-12", "Grupo B": "Miércoles 14-16", "Grupo C":
"Viernes 9-11"}
print(grupos[1])  # Muestra "Grupo B"
print(horarios["Grupo C"])  # Muestra "Viernes 9-11"
```

Duración sugerida: 2-2 horas

Taller de práctica Módulo 2:
1. Crea una lista con los nombres de tres grupos de tutores y usa `print` para mostrar el segundo grupo.
2. Define un diccionario que asocie cada grupo a una materia (ej. "Matemáticas", "Física") y muestra la materia del primer grupo.
3. Modifica la lista añadiendo un cuarto grupo y vuelve a imprimirla.

Respuestas orientativas:
1. `grupos = ["A", "B", "C"]; print(grupos[1])` muestra "B"
2. `materias = {"A": "Matemáticas"}; print(materias["A"])` muestra "Matemáticas"
3. `grupos.append("D"); print(grupos)` muestra la lista actualizada

Cierre del Módulo 2: Hemos visto cómo las condicionales dirigen el flujo lógico de los scripts y cómo listas y diccionarios permiten organizar y manipular conjuntos de datos. Estas herramientas son clave para crear programas que respondan a situaciones reales en la tutoría, como filtrar alumnos o gestionar horarios. El siguiente módulo introducirá la definición de funciones para modularizar código.

Módulo 3: Funciones y Módulos

Breve introducción al módulo: Aprenderás a encapsular código reusable en funciones y a aprovechar la biblioteca estándar de Python, lo que te permitirá organizar proyectos más complejos y ahorrar tiempo en tareas repetitivas.

3.1 Definición y reutilización de código

Objetivos de aprendizaje: Definir funciones con `def`, pasar parámetros y devolver valores con `return`, y aplicar el principio DRY (Don't Repeat Yourself) en scripts de tutoría.
Explicación desarrollada: Una función se define usando la palabra clave `followed by a name, parentheses, and a colon`. Parameters allow passing data into the function, and `return` sends a result back to the caller. Proper function design reduces redundancy, improves readability, and makes code easier to maintain across multiple courses or groups.
Puntos clave: Sintaxis `def nombre_funcion(parametros):`; `return` para salida de valores; conceptos de ámbito local y global básico.
Ejemplo práctico: Un tutor crea una función que genera un informe resumido de asistencia dado un diccionario de datos, y la reutiliza para diferentes cursos:

```
def informe_asistencia(datos):
    total = len(datos)
    presentes = sum(1 for v in datos.values() if v)
    return f"Total: {total}, Presentes: {presentes}"

grupo_a = {"Ana": True, "Ben": False, "Clara": True}
print(informe_asistencia(grupo_a))
```

Duración sugerida: 1-2 horas

3.2 Importación y uso de paquetes

Objetivos de aprendizaje: Importar módulos básicos de la biblioteca estándar (`math`, `random`, `sys`), utilizar `pip` para instalar paquetes adicionales y comprender el concepto de entornos virtuales para aislar proyectos.
Explicación desarrollada: La instrucción `import` accede a funcionalidades ya escritas en la biblioteca estándar de Python. Módulos comunes como `math` (operaciones matemáticas), `random` (generación de números aleatorios) y `sys` (interacción con el sistema) están disponibles sin instalación adicional. Para ampliar capacidades, `pip install`

`nombre_paquete` instala paquetes de terceros, y el uso de entornos virtuales (`.venv` o `conda`) mantiene las dependencias aisladas por proyecto, evitando conflictos entre diferentes cursos o herramientas.
Puntos clave: `import math`, `import random`, `import sys`; `pip install <paquete>`; creación de entorno virtual con `python -m venv nombre_entorno`.
Ejemplo práctico: Un tutor usa el módulo `random` para seleccionar al azar a un alumno para responder una pregunta en clase, o `sys` para leer argumentos de línea de comandos en un script de automatización:

```
import random
alumnos = ["Ana", "Ben", "Clara", "Diego"]
seleccionado = random.choice(alumnos)
print(f"Alumno seleccionado: {seleccionado}")
```

Duración sugerida: 1-2 horas

Taller de práctica Módulo 3:
1. Define una función `saludar(nombre)` que reciba un parámetro y devuelva la cadena `"Hola, nombre!"`. Llama a la función con tu nombre y muestra el resultado.
2. Usa el módulo `random` para elegir al azar un número entero entre 1 y 10 y muéstralo.

Respuestas orientativas:
1. `def saludar(nombre): return f"Hola, {nombre}!"; print(saludar("Tutor"))`
2. `import random; print(random.randint(1, 10))`

Cierre del Módulo 3: Las funciones permiten modularizar código, evitar repeticiones y estructurar proyectos de manera más limpia. Añadir la importación de módulos y el manejo de entornos virtuales amplía enormemente las posibilidades de automatización. En el módulo final veremos cómo aplicar todo lo aprendido a situaciones pedagógicas concretas con manejo de errores y automatización de tareas.

Módulo 4: Aplicaciones Pedagógicas

Breve introducción al módulo: Este módulo integra los conceptos previos para resolver situaciones reales de la práctica docente, centrándose en la robustez de los scripts y la automatización de procesos administrativos simples.

4.1 Manejo de errores básicos

Objetivos de aprendizaje: Utilizar bloques `try` y `except` para anticipar y gestionar errores comunes, y emplear `finally` para acciones de limpieza independientemente del resultado.
Explicación desarrollada: El manejo de errores evita que los scripts se cierren inesperadamente ante situaciones como archivos inexistentes, datos no esperados o entradas inválidas. `try` contiene el código que podría fallar, `except` captura tipos

específicos de excepciones y ejecuta acciones alternativas, y `finally` ejecuta bloques de limpieza siempre, útiles para cerrar archivos o liberar recursos.
Puntos clave: Estructura `try-except-finally`; captura de `FileNotFoundError`, `ValueError`; mensaje de error amigable para el usuario.
Ejemplo práctico: Un tutor escribe un script que lee un archivo de notas y usa `try/except` para manejar si el archivo no existe o los datos no son numéricos, mostrando un mensaje amigable:

```
try:
    with open("notas.txt", "r") as f:
        contenido = f.read()
        # Procesamiento simplificado
        print("Archivo leído correctamente")
except FileNotFoundError:
    print("El archivo de notas no se encontró. Verifica la ruta.")
except ValueError:
    print("Error al procesar los datos: formato inesperado.")
finally:
    print("Operación finalizada")
```

Duración sugerida: 1-2 horas

4.2 Automatización de tareas sencillas

Objetivos de aprendizaje: Combinar variables, bucles, condicionales, funciones y manejo de errores para automatizar una tarea administrativa repetida, como la generación de carpetas o archivos de informe.
Explicación desarrollada: La automatización integra los conceptos vistos a lo largo del curso. Un script puede recorrer una lista de grupos, crear una carpeta para cada uno usando `os.makedirs()`, generar un archivo de texto con datos básicos y registrar si el proceso completó exitosamente o encontró errores. Se recomienda probar el script en un directorio temporal antes de aplicarlo a estructuras reales.
Puntos clave: Integración de `for` loops, `os.makedirs()`, `open()` para escritura, `try-except` para robustez.
Ejemplo práctico: Un tutor automatiza la creación de carpetas y archivos de informe para cada uno de sus grupos, usando variables, bucles y manejo de errores básicos:

```
import os

grupos = ["Grupo A", "Grupo B", "Grupo C"]
for grupo in grupos:
    try:
        os.makedirs(grupo, exist_ok=True)
        ruta_archivo = os.path.join(grupo, f"informe_{grupo}.txt")
        with open(ruta_archivo, "w") as f:
            f.write(f"Informe para {grupo}")
        print(f"Carpeta y archivo creados para {grupo}")
    except Exception as e:
        print(f"Error al procesar {grupo}: {e}")
```

Duración sugerida: 1-2 horas

Taller de práctica Módulo 4:

1. Escribe un script que intente crear una carpeta llamada "Recursos_2024". Si la carpeta ya existe, que el script muestre un mensaje indicándolo en lugar de fallar.
2. Crea un diccionario con tres asignaturas y sus respectivas horas semanales. Intenta imprimir la hora de una asignatura inexistente usando `try/except` para capturar el error `KeyError`, y muestra un mensaje personalizado.

Respuestas orientativas:

1. `import os; try: os.makedirs("Recursos_2024", exist_ok=True); print("Carpeta lista o ya existía"); except Exception as e: print(f"Error: {e}")`
2. `asignaturas = {"Matemáticas": 6, "Física": 4}; try: print(asignaturas["Química"]); except KeyError: print("Asignatura no encontrada en el diccionario")`

Cierre del Módulo 4: Has integrado sintaxis, estructuras de control, funciones, módulos y manejo de errores para crear scripts robustos y automatizables. Estos recursos te permitirán ahorrar tiempo en tareas administrativas y diseñar herramientas a medida para tus contextos de tutoría. Con este módulo finalizas los contenidos técnicos básicos de Python; el próximo paso es aplicar estos proyectos en situaciones reales de tu práctica formativa.

```
```